DHARMA / DESARROLLO WEB

Plan de desarrollo de la tienda online

Una ruta gradual para conectar la landing, la tienda, el carrito, los pagos y el panel de administración.

Punto de partida	Dirección
Landing Vite existente; tienda y panel en etapa de estructura.	Un repositorio, módulos separados y una API de servidor para datos y pagos.
Catálogo y prendas por decidir.	No publicar productos ni precios ficticios.
Primera venta: carrito y cobro mediante API.	El backend valida productos, stock e importe antes de crear el cobro.

Versión 1.0 | 12 de noviembre de 2026

Documento de planificación. Los fragmentos de código ilustran ubicaciones y responsabilidades; no todos están implementados.

1. Resumen ejecutivo

Dharma ya tiene una landing en JavaScript vanilla con Vite y Tailwind CSS 4. Se prepararon entradas separadas para la tienda y una vista inicial del panel. El siguiente objetivo es construir la tienda en etapas pequeñas, conservando la landing como presentación de marca y llevando la compra a una ruta propia.

Recomendación inicial: mantener un solo repositorio y separar frontend y backend por carpetas. La tienda puede publicarse bajo /store/ y la API bajo /api/ en el mismo dominio, aunque cada parte se despliegue como servicio distinto. Así se conserva una experiencia integrada sin mezclar el código de interfaz, pagos y datos.

La primera versión debe permitir una compra simple y una consulta administrativa de ventas por ID y fecha. El catálogo, los precios y los envíos siguen pendientes de definición; por eso el desarrollo inicial puede avanzar con contratos y pantallas vacías, pero no debe aceptar compras reales hasta que exista información comercial y la API de pagos esté validada.

Alcance recomendado para abrir

- Landing institucional con enlace a la tienda; la compra ocurre en /store/.
- Catálogo solo cuando existan prendas, variantes, precios e inventario reales.
- Carrito de visitante, sin cuenta obligatoria en la primera versión.
- API de servidor que crea pedidos pendientes y sesiones de pago.
- Panel protegido con historial mínimo de ventas por identificador y fecha.
- Envíos, cuentas de cliente, analítica avanzada y edición de contenido quedan para etapas posteriores.

Fuera del primer corte

No se incluyen datos de productos ficticios, tarifas inventadas, tracking de envíos, cuentas de clientes, gestión de contenido de la landing ni expansión internacional. Cada función se incorpora cuando Dharma defina sus reglas y exista una fuente de datos segura.

2. Diagnóstico del repositorio

Área	Estado observado	Qué falta
Landing	Vite, JavaScript vanilla, CSS/Tailwind y secciones modulares.	Mantener identidad y navegación; añadir el acceso a la tienda.
Tienda	Existe store/index.html y una pantalla de preparación.	Catálogo conectado a fuente de datos y flujo de compra.
Carrito	El encabezado muestra un contador visual estático.	Estado real, persistencia, ajuste de unidades y validación de stock.
API	Existe store/src/servicios/api.js vacío.	Servidor, endpoints, base de datos y validaciones.
Administración	Hay una pantalla de preparación separada.	Inicio de sesión seguro y consulta a ventas reales.
Pagos	No existe proveedor conectado.	Elegir pasarela y crear flujo sandbox/backend/webhook.

3. Arquitectura recomendada

Mantener el repositorio Git actual reduce coordinación y facilita aprender el flujo completo. Las carpetas separan la tienda y la API; en producción pueden compartir dominio aunque corran como servicios distintos.

Alternativa	Ventajas	Costos y recomendación
Un repositorio; frontend y API separados	Cambios coordinados, una sola historia de Git y aprendizaje de JavaScript en ambas capas.	Requiere límites claros entre navegador y servidor. Recomendada.
Repositorios separados	Despliegues y permisos independientes.	Más configuración y coordinación. Considerar si crecen equipos o ritmos.
Tienda alojada en plataforma externa	Menos operación propia al comenzar.	Menos control del flujo; no coincide con el objetivo actual.

Mapa de rutas

```
/                       landing Dharma
/store/                 tienda para visitantes
/store/admin/           interfaz de administración
/api/products           catálogo publicado
/api/checkout-sessions  pedido pendiente y sesión de pago
/api/payments/webhook   confirmación auténtica del proveedor
/api/admin/sales        consulta protegida por ID y fecha
```

El navegador no debe conectarse directamente a la base de datos ni guardar secretos de pago. La ruta visual del panel no lo protege; cada endpoint administrativo valida sesión y rol en el servidor.

Estructura de carpetas propuesta

```
/home/fabian/proyecto-personal/front/  # raíz Git existente
  index.html                           # landing Dharma
  src/                                 # secciones y estilos de landing
  store/
    index.html                         # entrada de tienda
    admin/index.html                   # entrada del panel
    src/
      componentes/                     # piezas visuales reutilizables
      datos/producto.js                # contrato de producto existente
      paginas/                         # storefront y administración
      servicios/carrito.js             # servicio local propuesto
      servicios/api.js                 # navegador -> API
  api/                                 # backend propuesto; aún no creado
    src/rutas/                         # catálogo, checkout, pagos, ventas
    src/servicios/                     # pedidos, pagos y ventas
    src/seguridad/                     # identidad y permisos
    migrations/                        # esquema versionado
    tests/                             # pruebas del backend
```

La carpeta `api/` se creará dentro de la raíz Git cuando se aprueben framework, alojamiento y base de datos. No se colocan claves privadas en `store/` ni en variables que Vite exponga al navegador.

4. Responsabilidades por archivo

Archivo o carpeta	Responsabilidad	Por qué va ahí
src/	Landing y narrativa de Dharma.	Separa la marca institucional de la compra.
store/index.html	Punto de entrada del storefront.	Publica la tienda bajo `/store/` y compila como página multipágina de Vite.
store/admin/index.html	Punto de entrada visual del panel.	Mantiene administración fuera de la compra; la seguridad vive en backend.
store/src/componentes/	Header, cards, formularios y estados vacíos.	Reutiliza UI sin mezclar contenido de páginas.
store/src/paginas/tienda.js	Composición de la página de tienda.	Arma la pantalla con componentes y datos.
store/src/paginas/administracion.js	Composición del historial de ventas.	Mantiene panel y storefront separados.
store/src/datos/producto.js	Contrato `Producto` y `VarianteProducto`.	Documenta el formato; no crea prendas.
store/src/servicios/carrito.js	Agregar, cambiar cantidad, quitar y persistir líneas.	La lógica no depende del HTML y se revisa aparte.
store/src/servicios/api.js	Llamadas `fetch` a endpoints de backend.	Centraliza URLs, errores y respuestas.
api/src/rutas/	Recibe requests, valida y responde.	Conecta HTTP con la lógica del dominio.
api/src/servicios/	Crea pedidos, solicita pagos y consulta ventas.	Mantiene la lógica sensible en el servidor.
api/src/seguridad/	Autenticación y permisos administrativos.	No confía en botones ocultos ni en la URL.
api/migrations/	Esquema de base de datos versionado.	Permite reconstruir y actualizar tablas.

Ejemplo del contrato de producto

Este contrato está en `store/src/datos/producto.js`. Describe la forma de los datos, pero no crea prendas ni sustituye la validación del backend.

```
/** Variante vendible identificada por SKU. */
/** @typedef {Object} VarianteProducto
 * @property {string} sku
 * @property {string} talla
 * @property {string} color
 * @property {number} unidadesDisponibles
 */

/** Información común de una prenda aprobada. */
/** @typedef {Object} Producto
 * @property {string} id
 * @property {string} nombre
 * @property {string} categoria
 * @property {number} precioCLP
 * @property {string[]} imagenes
 * @property {VarianteProducto[]} variantes
 * @property {boolean} publicado
 */
```

La disponibilidad no se inventa: se integra cuando Dharma defina una fuente de inventario confiable.

5. Datos y API

El pedido conserva una copia del nombre, variante, precio y cantidad comprados. El modelo es conceptual, no un esquema SQL final. La disponibilidad solo se publica cuando existe una fuente de inventario acordada; no se abren ventas mientras falten prendas, precios, stock confiable y condiciones de entrega.

Entidad	Campos y responsabilidad	Nota
products	ID, nombre, categoría, descripción, `precioCLP`, imágenes y publicado.	Datos aprobados para mostrar y precio vigente a nivel de producto.
product_variants	SKU, product_id, talla, color y unidades disponibles.	La fuente de inventario está pendiente; no se asume sincronización de dropshipping.
orders	ID, createdAt, paidAt nullable, estado, total, moneda y referencia de pago.	paidAt se asigna al confirmar el pago y sirve como fecha de venta.
order_items	Pedido, producto y variante en snapshot, precio unitario CLP y cantidad.	Conserva lo que se compró aunque cambie el catálogo.
payment_events	Proveedor, ID externo, estado, monto, event_id y recibido_en.	Permite conciliar avisos y procesarlos una sola vez.
admin_users	Identidad protegida, rol y activo.	La identidad y recuperación del administrador siguen pendientes.

El panel administrativo muestra solo `id` y `paidAt` (fecha de venta) de pedidos pagados. Estado, monto y datos de pago se conservan para operar el backend; no se incluyen en la respuesta del historial. Direcciones u otros datos de despacho se incorporan cuando se definan las reglas de envío.

Endpoints mínimos propuestos

Método y ruta	Quién	Propósito
GET /api/products	Visitante	Solo productos publicados con disponibilidad respaldada por la fuente de inventario acordada.
POST /api/checkout-sessions	Visitante	Valida IDs, SKU, unidades, precio y stock; crea pedido pendiente y checkout.
POST /api/payments/webhook	Proveedor	Valida autenticidad del evento y evita procesarlo dos veces.
GET /api/admin/sales?id=&from=&to=	Administrador autenticado	Devuelve únicamente ID y paidAt de pedidos pagados.

Flujo seguro del cobro

• El navegador envía IDs de producto, SKU de variante y cantidades; no envía un total confiable.

• La API carga precio y disponibilidad desde fuentes aprobadas, valida y calcula el total.

• La API crea un pedido pendiente y solicita la sesión de pago con credenciales del servidor.

• El navegador redirige al checkout alojado por el proveedor.

• El webhook auténtico confirma el pago; la API valida el evento y su idempotencia.

• La API asigna paidAt al confirmar pago. El retorno del navegador no confirma por sí solo una venta.

Las credenciales de prueba y producción se mantienen separadas y nunca se envían al frontend.

6. Seguridad del panel y datos

Permisos iniciales

Rol	Puede hacer	Límite
Visitante	Ver landing y tienda; iniciar checkout como invitado.	No ve pedidos de otras personas ni administración.
Administrador	Consultar ventas por ID/fecha según el alcance inicial.	El servidor valida sesión y permisos en cada endpoint.
Cliente frecuente	Segmento futuro de analítica.	No obtiene permisos adicionales.

Controles a implementar antes de datos reales

- Autenticación de administrador en backend; no basta un enlace secreto, una ruta oculta o meta noindex.
- Validar formato, cantidad, producto, stock y precios en servidor.
- Verificar firma del webhook, registrar eventos y procesar cada evento una sola vez.
- Usar HTTPS y secretos de entorno exclusivamente en el servicio backend.
- Limitar intentos de acceso, proteger sesiones y registrar actividad administrativa relevante.
- Recolectar solo datos necesarios para compra y despacho; definir conservación y acceso.
- Respaldar la base de datos y probar recuperación antes de abrir ventas.

Chile y cumplimiento

La tienda parte en Chile. Antes de aceptar pedidos reales se debe revisar la normativa vigente en fuentes oficiales de BCN, SERNAC y SII y validar con un profesional las obligaciones aplicables a Dharma. El plan no afirma obligaciones legales específicas ni reemplaza asesoría. La matriz para Estados Unidos, Unión Europea y China continental se prepara solo si Dharma decide vender o dirigir actividad a esos mercados.

Analítica y privacidad

En el lanzamiento se puede medir lo indispensable: visitas agregadas, productos vistos, inicio de checkout, compra pagada, monto y despacho. Antes de instalar herramientas de terceros hay que definir finalidad, datos enviados, retención, consentimiento o configuración requerida. El dato de cliente frecuente se define como segmento comercial, nunca como rol de acceso.

7. Etapas de desarrollo

Etapa	Trabajo y resultado	Estado / condición
0. Base del repo	Landing existente; entradas separadas de tienda y panel; documentos de contexto.	Parcial: las vistas están vacías y el panel no es seguro.
1. Reglas de la tienda	Acordar visitante/cliente, compra invitada, variantes, cambios y estados.	Pendiente de acuerdos de negocio.
2. Catálogo y variantes	Definir datos, API de lectura y stock; cargar prendas auténticas.	Pendiente del catálogo y los precios aprobados.
3. Carrito	Agregar, ajustar, quitar, persistir y gestionar errores de stock.	Propuesta de servicio presentada; espera revisión del usuario.
4. Backend y base de datos	Crear API, migraciones, pedidos y snapshots de producto.	Pendiente elegir stack y alojamiento.
5. Pasarela	Construir el pago sandbox y su webhook con el proveedor elegido.	Comparación documentada; falta elegir proveedor.
6. Panel mínimo	Login protegido; filtrar por ID/fecha; mostrar solo ID y fecha de venta.	Pendiente de API y autenticación.
7. Validación y apertura	Pruebas de flujo, permisos, pagos duplicados, responsive, respaldo y publicación.	Antes de recibir dinero real.
8. Envíos y cuentas	Definir abastecimiento, despacho, plazos y consulta de pedidos por cliente.	Fase posterior; requiere reglas y tarifas definidas.
9. Analítica y expansión	KPIs, privacidad por mercado, automatización y otros países.	Solo cuando haya necesidad comercial.

Orden práctico recomendado

Primero cerrar el contrato de carrito y su revisión. En paralelo, acordar el modelo de abastecimiento, qué significa una venta registrada y quién administrará la tienda. Luego seleccionar backend, base de datos y pasarela con documentación oficial. Sin prendas aprobadas, fuente de stock, precios y condiciones de entrega, no habilitar una compra real.

Cómo aprender en cada etapa

- Antes de tocar código: objetivo, conceptos nuevos, archivos y criterios de terminado.
- Después del cambio: explicar funciones y datos, mostrar cómo probar manualmente y resumir lo aprendido.
- Revisar Git y el diff; nunca reemplazar archivos existentes sin avisar.
- Mantener los cambios pequeños y aprobar el código antes de integrarlo.

8. Decisiones pendientes

Decisión	Por qué se necesita	Cuándo resolverla
Prendas, variantes, precios y stock	Determinan catálogo, totales y compra real.	Antes de publicar ventas.
Compra con o sin cuenta	Define datos de cliente, sesiones y consulta de pedidos.	Al cerrar alcance MVP.
Pasarela disponible en Chile	Define creación de sesión, webhook, comisiones, reembolsos y conciliación.	Antes de implementar pagos; verificar fuente oficial.
Framework y alojamiento del backend	Determinan API, despliegue, logs y secretos.	Antes de crear api/.
Base de datos y proveedor	Define persistencia, respaldo y residencia de datos.	Antes de guardar ventas reales.
Acceso administrador	Define identidad, segundo factor, recuperación y auditoría.	Antes de exponer el panel.
Despacho y precio de envío	El cliente debe conocer el total antes de pagar.	Antes de habilitar checkout.
Dominio y rutas de producción	Permite servir /, /store/ y /api/ con HTTPS.	Antes de publicar.

Criterios de salida de la primera versión

- El catálogo contiene datos y fotos que Dharma aprobó, con stock y precio vigentes.
- Un visitante puede elegir variante, revisar unidades y conocer el total completo antes del pago.
- La API recalcula el importe, crea el pedido pendiente y llama a la pasarela en sandbox.
- El servidor actualiza el pago solo tras confirmación auténtica y tolera eventos repetidos.
- Un administrador autenticado puede filtrar el historial por ID y fecha.
- No se filtran pedidos de clientes ni datos privados a usuarios no autorizados.
- La tienda funciona en móvil y escritorio; hay registro de errores y respaldo recuperable.

Términos útiles

Término	Explicación sencilla
API	Contrato por el que la tienda pide datos o acciones al servidor.
Backend	Programa que valida reglas, conecta base de datos y habla con la pasarela.
Webhook	Aviso servidor-a-servidor que informa el resultado de un pago.
Idempotencia	Regla para procesar el mismo aviso más de una vez sin duplicar la venta.
Snapshot	Copia de nombre, variante, cantidad y precio que se conserva dentro del pedido.
SKU	Código de una variante concreta, por ejemplo combinación de prenda y talla.

Documento base para conversar y aprender. Las decisiones marcadas como pendientes se revisan antes de implementarlas.

Anexo: pasarelas para pagos online en Chile

Consulta de fuentes oficiales realizada el 12 de noviembre de 2026. Los datos de tarifas y afiliación deben volver a verificarse antes de contratar.

Aspecto	Mercado Pago Checkout Pro	Transbank Webpay Plus
Flujo	API del servidor crea una order y devuelve checkout_url. El comprador paga en checkout alojado; admite invitado.	API del comercio crea la transacción y redirige al formulario Webpay. El servidor confirma el resultado con el token.
Medios publicados	Crédito, débito, prepago, transferencia y saldo Mercado Pago.	Crédito, débito y prepago.
Seguridad / notificación	Webhook HTTPS con firma secreta; X-Idempotency-Key en creación de order.	Retorno con token y confirmación de servidor; validar el estado y response_code antes de marcar pagado.
Tarifa publicada al consultar	3,19% + IVA con disponibilidad al instante; 2,89% + IVA en 10 días. Se muestran tasas reducidas para cuentas nuevas. Confirmar oferta de Dharma.	Sin mensualidad fija según la página del producto; comisión por venta depende del negocio. Usar el simulador o tarifa de la cuenta.
Abono indicado	Depende del plazo y modalidad elegidos; confirmar condiciones de la cuenta.	24 horas hábiles débito/prepago; 48 horas hábiles crédito, según página de producto.

Lectura para Dharma

Mercado Pago Checkout Pro es un candidato para prototipo por su URL de checkout alojado, compra invitada y variedad de medios publicados. Webpay Plus es candidato para cobros con tarjetas en una pasarela chilena. No se selecciona un ganador: primero se confirman afiliación, comisión, plazo de abono, reembolsos, contracargos y soporte para Dharma.

En cualquiera de las dos opciones, la API de Dharma calcula el total y guarda el pedido pendiente. La credencial privada permanece en backend. El pago se marca como aprobado después de verificar una confirmación auténtica, con protección frente a eventos duplicados.

Fuentes oficiales

Mercado Pago: resumen de Checkout Pro · crear order e idempotencia · webhooks y firma · tarifas publicadas de Mercado Pago.

Transbank Developers: flujo Webpay Plus · producto, medios y abono · tarifas y comisiones.

No es una cotización ni asesoría contable, legal o financiera. Las condiciones pueden variar por cuenta, contrato y fecha.

Actualización de datos y ventas

Anexo técnico · 12 de noviembre de 2026 · Actualiza la página 5 del plan

Contrato de producto

Producto: ID, nombre, categoría, `precioCLP`, imágenes, variantes y publicación. El precio pertenece al producto según el JSDoc aprobado.

Variante: SKU, talla, color y unidades disponibles. El contrato actual no define un precio distinto por variante.

Línea de pedido: conserva nombre, variante, precio unitario CLP y cantidad al momento de compra.

Qué fecha muestra el panel

En el modelo de pedido se mantienen `createdAt` para el intento y `paidAt` para la venta confirmada. El backend asigna `paidAt` solo al validar el estado con el proveedor; un reintento de webhook no cambia esa fecha.

> **Historial inicial:** muestra únicamente `id` y `paidAt` de pedidos pagados. La búsqueda por fecha usa `paidAt`; pedidos pendientes, rechazados o vencidos no se muestran como ventas.

Confirmación y pasarelas

La comparación oficial de Mercado Pago Checkout Pro y Transbank Webpay Plus está documentada en `docs/tienda/PASARELAS-CHILE.md`. En la API de Dharma, los estados del proveedor se normalizan; solo una respuesta autoritativa puede marcar un pedido como pagado. La selección de proveedor sigue pendiente.

> **Alcance:** no hay productos, precios ni ventas reales definidos. El backend, la base de datos y el acceso seguro del panel aún no están implementados.

Raíz del proyecto y despliegue

Anexo actualizado · 12 de noviembre de 2026

Estructura confirmada

La raíz Git real es **/home/fabian/proyecto-personal/front/**. Allí conviven la landing existente y la aplicación de tienda. Las rutas del plan se interpretan desde esa raíz.

```
proyecto-personal/front/   (raíz Git actual)
- index.html              landing Dharma
- src/                     interfaz de marca
- store/                   tienda y panel iniciales
- api/                     backend propuesto; aún no creado
```

Despliegue propuesto

La landing, la tienda y la API pueden desplegarse por separado. La arquitectura pública prevista conserva las rutas **/**, **/store/** y **/api/**; la ubicación física de los servicios dependerá del alojamiento elegido.

Decisiones todavía pendientes

Componente	Estado
Backend	Fastify 5 es una recomendación técnica, aún no aprobada ni instalada.
Datos y alojamiento	Base de datos, proveedor de hosting y forma de despliegue por definir.
Inventario y pagos	Abastecimiento, proveedor de pago y disponibilidad de productos por confirmar.

Estado: la carpeta **api/** y sus endpoints siguen como propuesta. No hay proveedor de pago, base de datos, autenticación ni fuente de inventario configurados.

ACTUALIZACIÓN · 12 DE NOVIEMBRE DE 2026

Aclaraciones para el plan de desarrollo

Esta página actualiza el estado de las pasarelas y deja explícita una decisión de negocio aún abierta.

Investigación oficial de pasarelas completada

La comparación inicial de Mercado Pago Checkout Pro y Transbank Webpay Plus está documentada en la página 9 y en docs/tienda/PASARELAS-CHILE.md, con enlaces a fuentes oficiales.

La etapa 5 de la página 7 ya no está pendiente de investigación. Sigue pendiente elegir proveedor y confirmar afiliación de Dharma, comisiones aplicables, plazos de abono, reembolsos, contracargos y soporte.

Abastecimiento y despacho sin definir

Se consideró usar dropshipping temporal mientras Dharma consolida qué prendas producir, pero el modelo no está confirmado. No se han definido proveedor, fuente de inventario, plazos ni responsabilidades de despacho.

No se deben anunciar stock o fechas de entrega ni habilitar ventas reales hasta acordar esas condiciones. Las pantallas, el contrato del carrito y otras estructuras pueden prepararse mientras tanto, sin inventar disponibilidad ni promesas de entrega.

SITUACIÓN DEL PROYECTO

No hay pasarela seleccionada, catálogo real, stock aprobado ni pagos conectados. El modelo de abastecimiento y la selección del proveedor de pago son decisiones distintas; ambas deben resolverse antes de abrir ventas.

Demo local y ruta hasta producción

Anexo de estado · 12 de noviembre de 2026

Qué entrega la demo actual

Rutas: landing **/**, tienda **/store/** y administración demo **/store/admin/**. La tienda inicia sin productos precargados.

El panel permite crear, editar, publicar, ocultar y eliminar productos con variantes. El catálogo y el carrito se guardan en el `localStorage` del mismo navegador. La tienda calcula un subtotal referencial sin despacho; el botón de cobro está desactivado y el historial permanece vacío.

Límite de seguridad: no hay backend, base de datos ni autenticación. Los valores ingresados son datos de demostración, no inventario validado. No usar el panel para datos de clientes ni aceptar pagos.

Decisiones y etapas que faltan para vender

Etapa	Requisito previo o entregable	Condición para avanzar
1 · Definir la oferta	Aprobar prendas, precios, tallas/colores, fuente de inventario, abastecimiento y despacho.	No publicar inventario o plazos sin una fuente y responsabilidades confirmadas.
2 · Elegir plataforma	Seleccionar framework, base de datos, alojamiento y proveedor de pago. Fastify 5 es solo una recomendación pendiente.	Fijar Node y secretos de servidor antes de crear el entorno de producción.
3 · Construir backend	API de catálogo y checkout, cálculo de precio/stock en servidor, pedidos, idempotencia y confirmación por webhook.	El retorno del navegador no confirma el pago. Solo el servidor registra una venta confirmada.
4 · Proteger administración	Autenticación y permisos; edición segura de catálogo; historial por ID y fecha de pago.	La consulta inicial devuelve solo ID y fecha de venta, como se solicitó.
5 · Preparar lanzamiento	Pruebas de compra y errores, HTTPS, backups, privacidad, monitoreo y costos finales de despacho.	Habilitar pagos reales solo con oferta, precio total y operación aprobados.

El contrato propuesto, rutas y validaciones están detallados en `docs/tienda/API-DATOS.md`. La demo local facilita revisar la presentación y administrar datos de prueba en un solo navegador; reemplazar `localStorage` por servicios protegidos es una etapa distinta.